

Decoders

Tópicos em Fundamentos e Métodos da Computação

Prof. Dr. Luís Paulo Faina Garcia



Universidade de Brasília

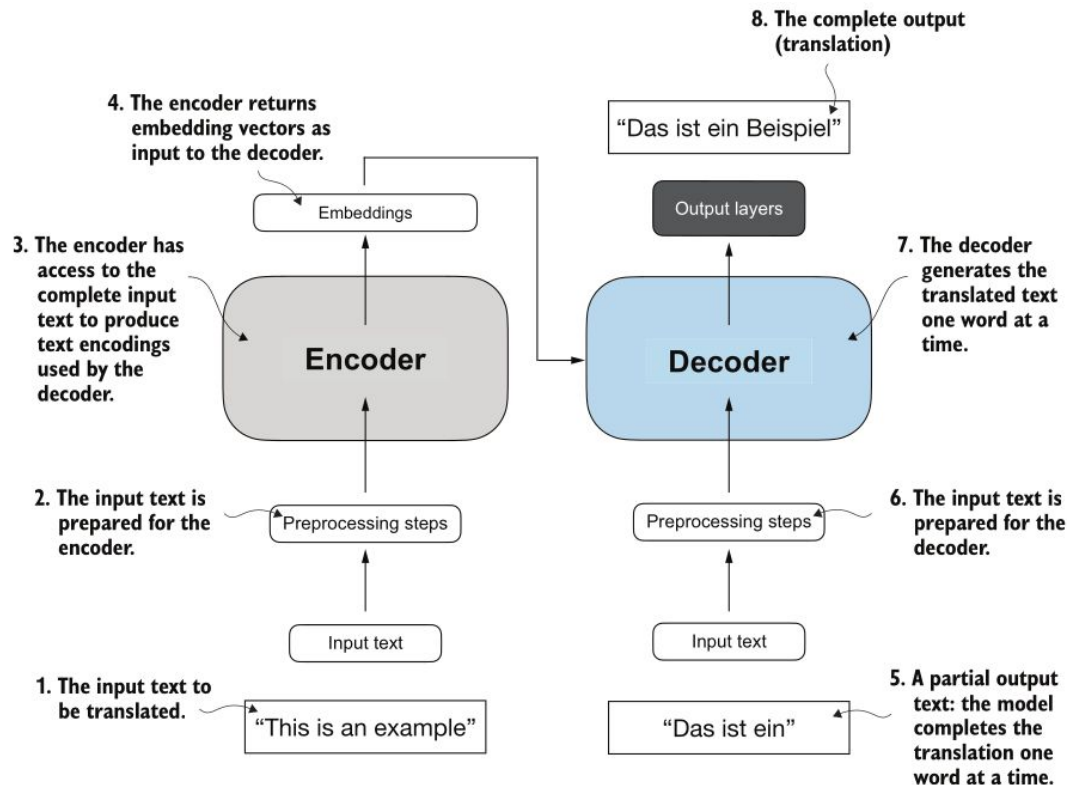
Departamento de Ciência da Computação

Transformer Architecture

Originally designed for sequence-to-sequence tasks (like machine translation). This architecture consists of two components:

- **Encoder:** Converts input tokens into a sequence of embedding vectors (hidden state/context);
- **Decoder:** Uses encoder state to generate output tokens.

Transformer Architecture



Transformer Architecture

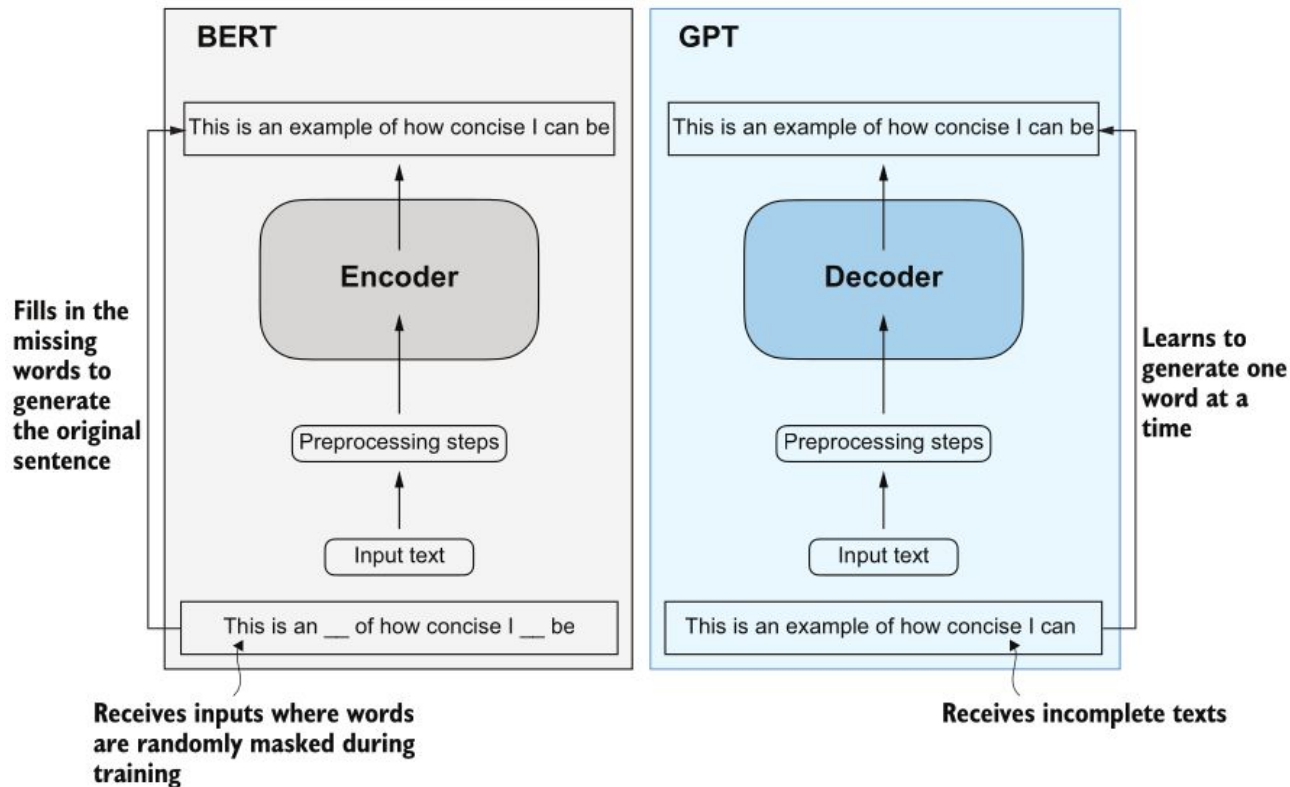
- The transformer architecture consists of two submodules: an encoder and a decoder;
- The encoder module processes the input text and encodes it into a series of numerical representations that capture the contextual information of the input;
- The decoder module takes these encoded vectors and generates the output text;

Transformer Architecture

Later, variants of the transformer architecture, such as BERT and the various GPT models (Generative Pretrained Transformers), built on this concept to adapt this architecture for different tasks:

- BERT is specialized in masked word prediction*
- GPT is designed for generative tasks*

Transformer Architecture



Zooming the GPT

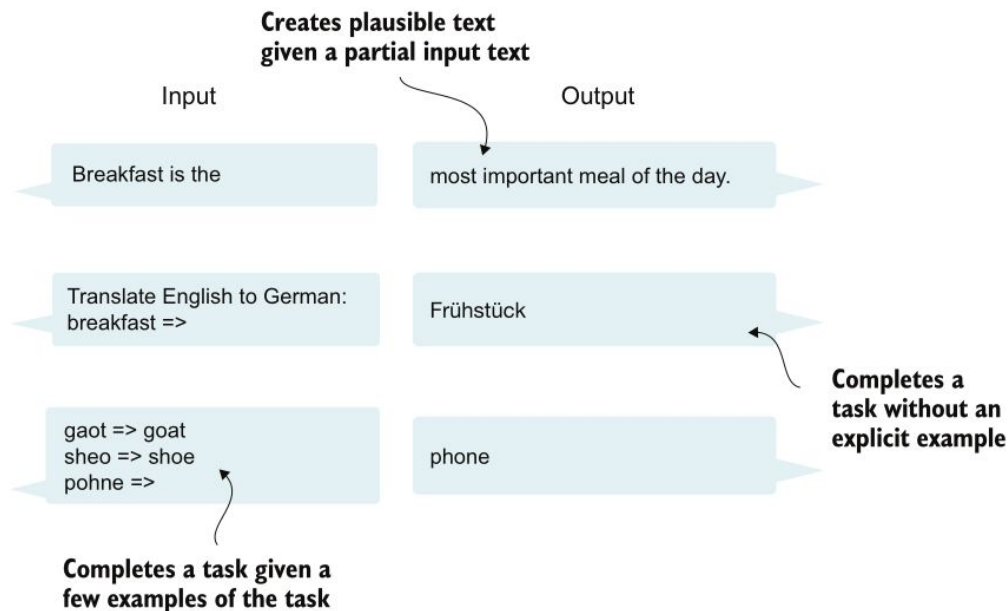
GPT models are pre trained on a relatively simple next-word prediction task:

**The model is simply trained to
predict the next word**



Zooming the GPT

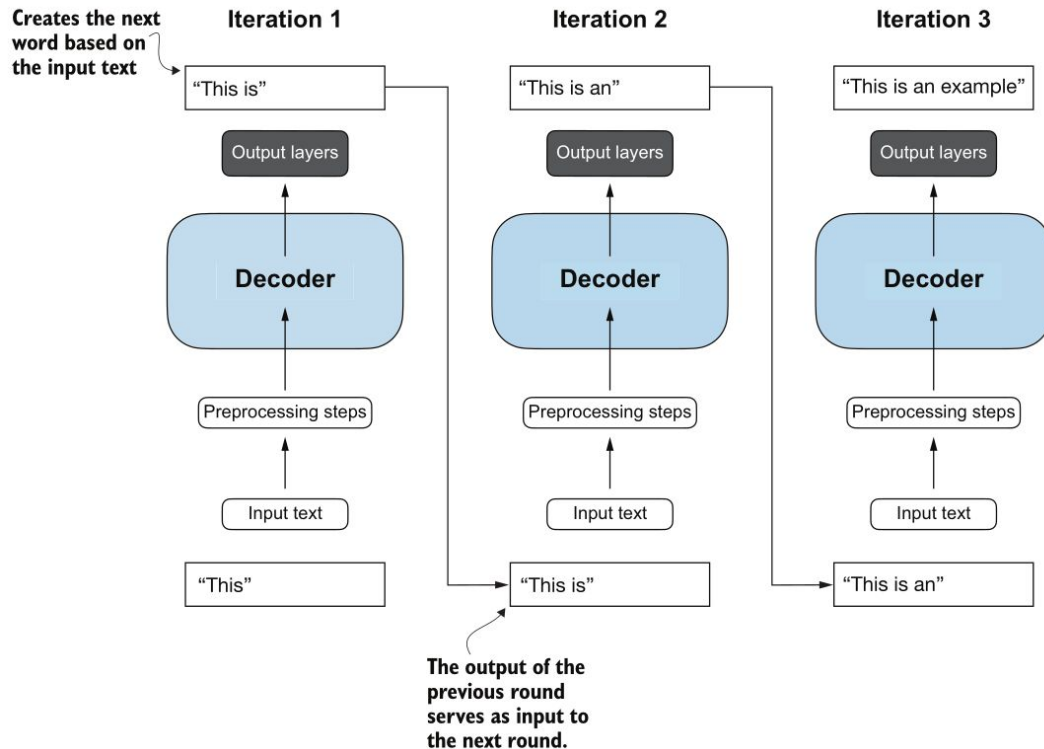
GPT models are pre trained on a relatively simple next-word prediction task:



Zooming the GPT

- Since decoder-style models like GPT generate text by predicting text one word at a time, they are considered a type of autoregressive model.
- Autoregressive models incorporate their previous outputs as inputs for future predictions.

Zooming the GPT



Decoders

The main difference between the decoder and encoder is that the decoder has a masked multi-headed self-attention layer:

- Ensures that the tokens we generate at each timestep are only based on the past outputs and the current token being predicted;
- Without this, the decoder could cheat during training by simply copying the target translations.

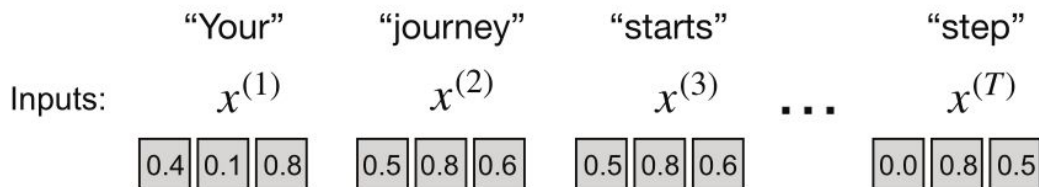
Self-attention

- In self-attention, the “self” refers to the mechanism ability to compute attention weights by relating different positions within a single input sequence;
- It assesses and learns the relationships and dependencies between various parts of the input itself;
- The goal of self-attention is to compute a context vector (an enriched embedding vector).

Attention Mechanism without Trainable Parameters

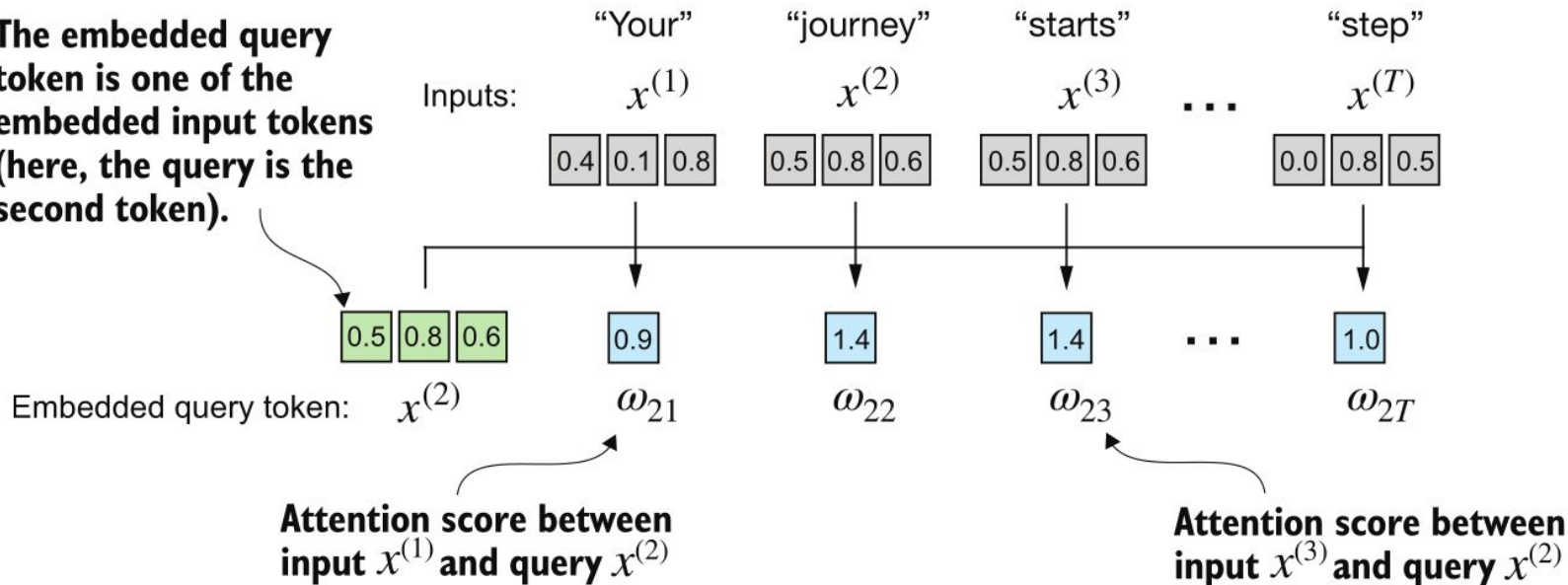
The goal of self-attention is to compute a context vector for each input element that combines information from all other input elements.

Attention Mechanism without Trainable Parameters

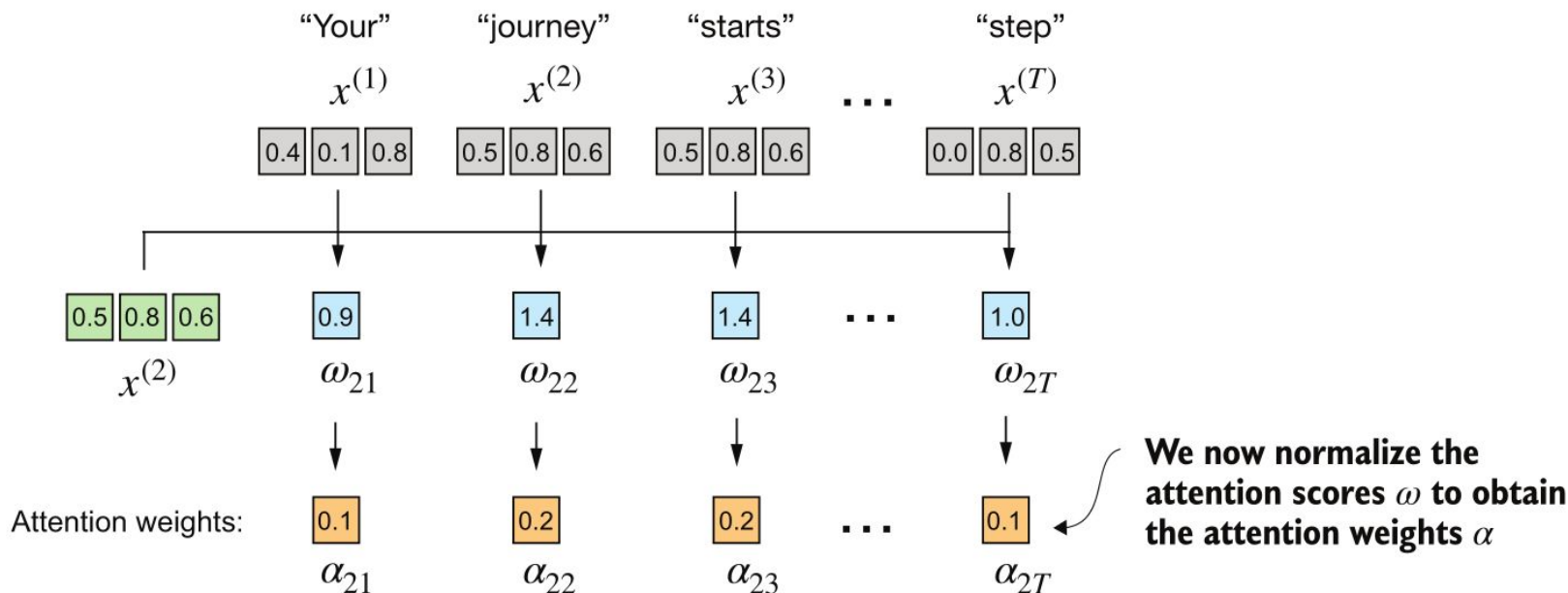


Attention Mechanism without Trainable Parameters

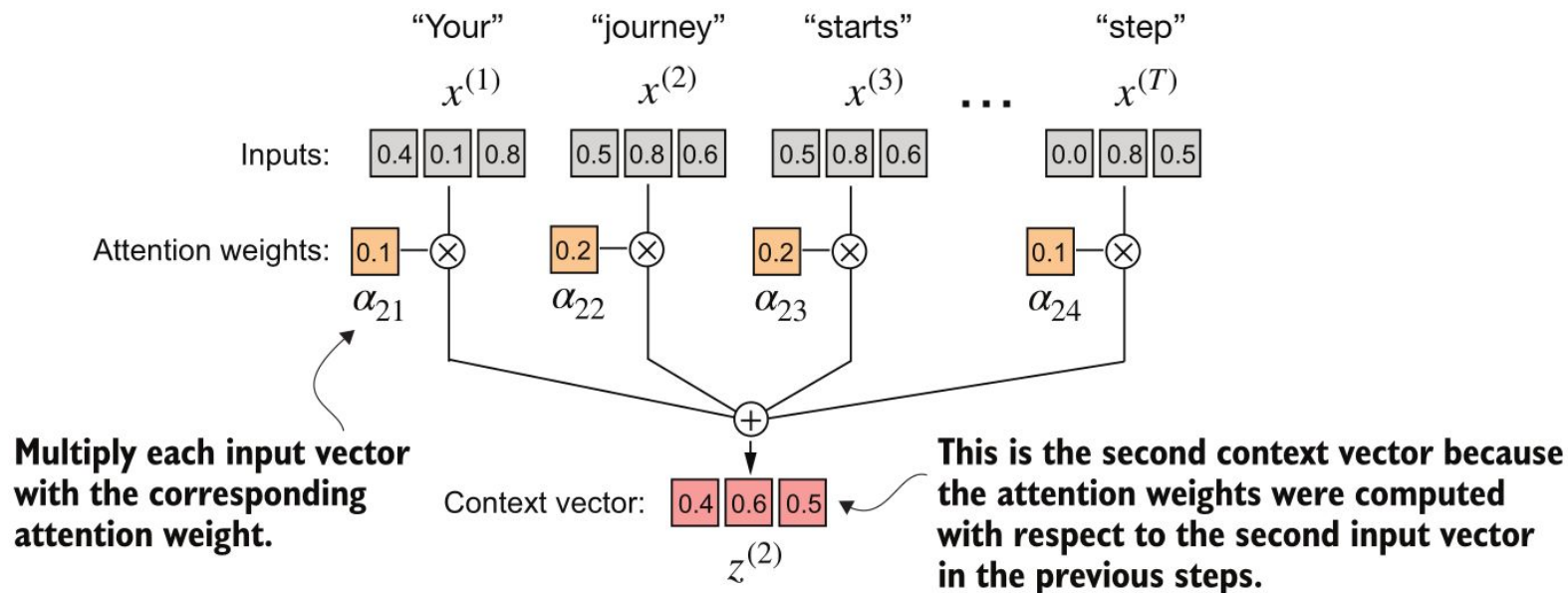
The embedded query token is one of the embedded input tokens (here, the query is the second token).



Attention Mechanism without Trainable Parameters

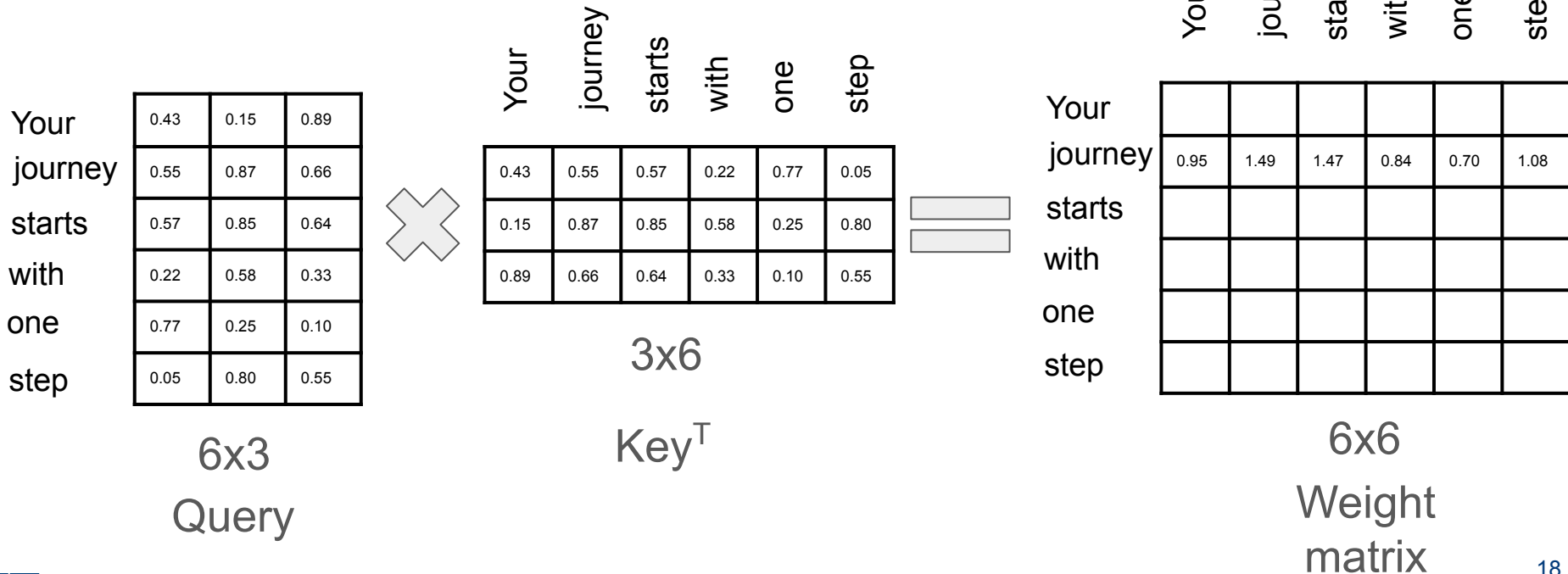


Attention Mechanism without Trainable Parameters



Attention Mechanism without Trainable Parameters

Matrix multiplication



Attention Mechanism without Trainable Parameters

Scale + softmax

$$\frac{1}{\sqrt{d}} QK^{\top}$$

$$\text{Softmax}\left(\frac{QK^{\top}}{\sqrt{d}}\right)$$

0.54	0.86	0.84	0.48	0.40	0.62

Apply Softmax(.)
on each row



0.13	0.23	0.23	0.12	0.10	0.15

Attention Mechanism without Trainable Parameters

Scale + softmax

Without this adjustment, the dot product results in very large values.

$$\frac{1}{\sqrt{d}} QK^T$$

$$\text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)$$

0.54	0.86	0.84	0.48	0.40	0.62

Apply Softmax(.)
on each row



0.13	0.23	0.23	0.12	0.10	0.15

Attention Mechanism without Trainable Parameters

Matrix multiplication

0.13	0.23	0.23	0.12	0.10	0.15

6x6

Attention
matrix



Your
journey
starts
with
one
step

0.43	0.15	0.89
0.55	0.87	0.66
0.57	0.85	0.64
0.22	0.58	0.33
0.77	0.25	0.10
0.05	0.80	0.55

6x3

Value



Context
matrix

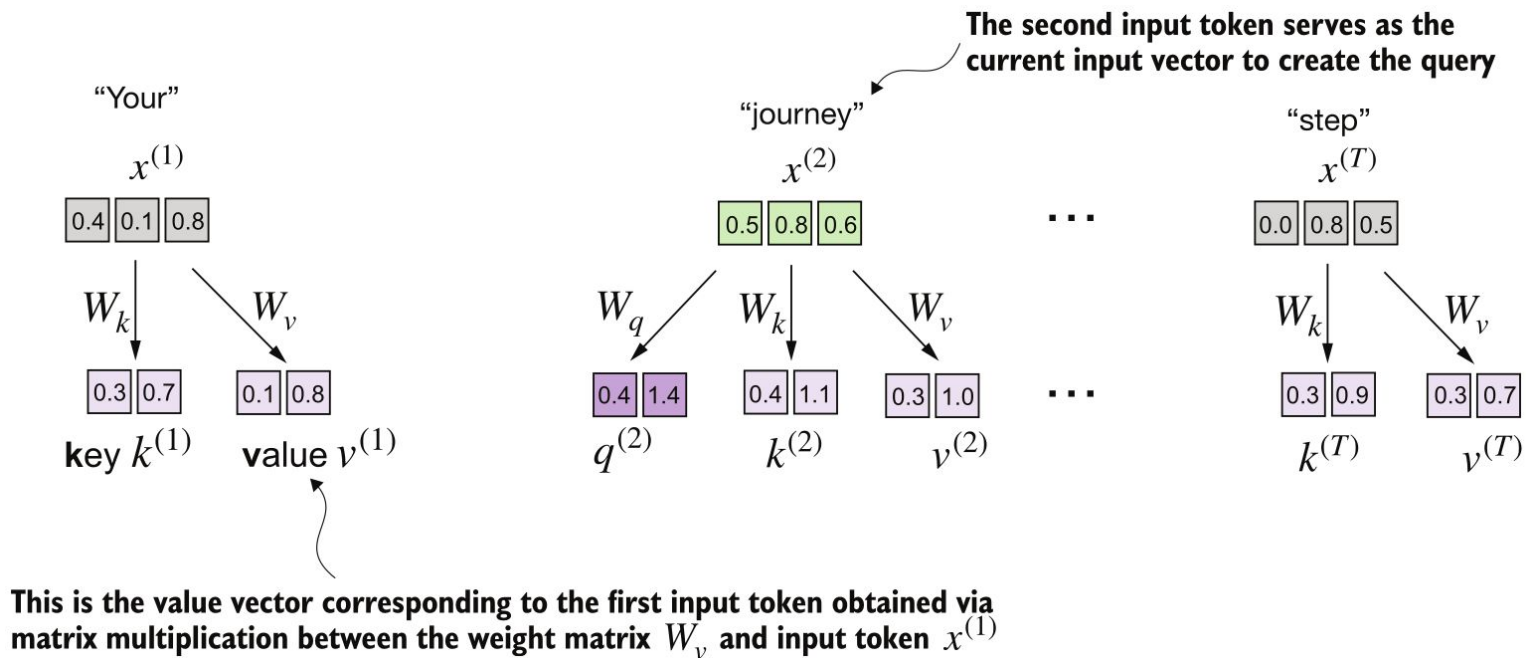
Attention Mechanism with Trainable Parameters

Trainable parameters means the introduction of weight matrices that are updated during model training;

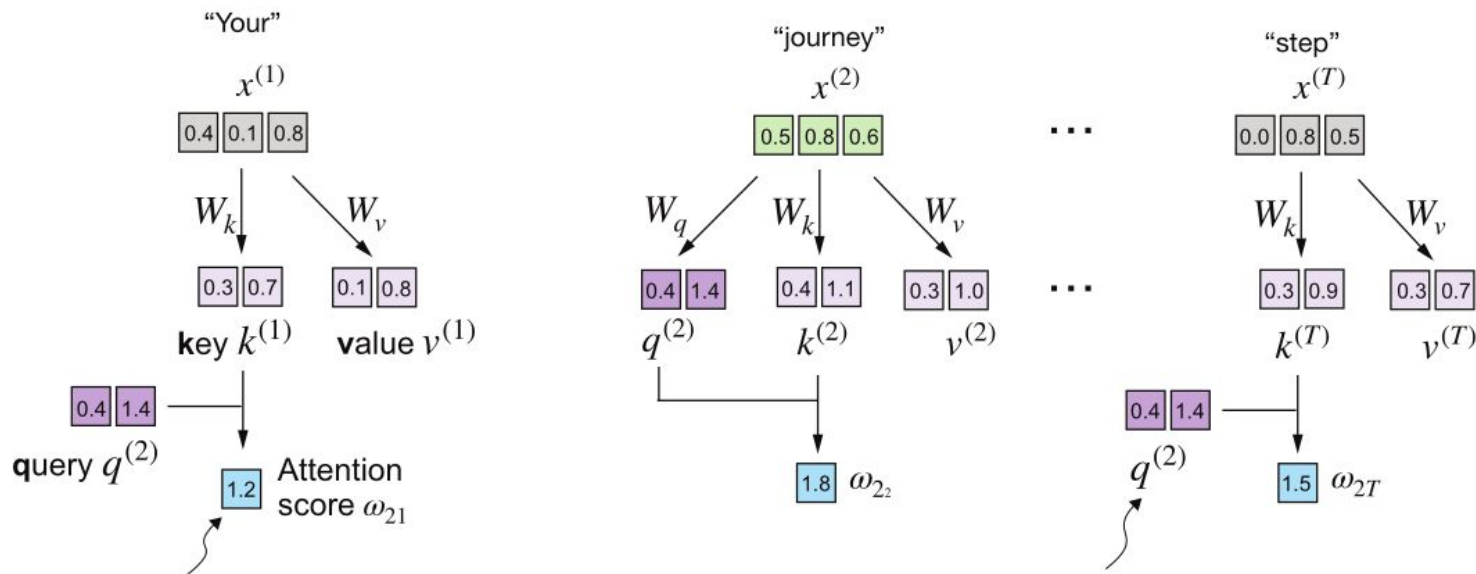
These trainable weight matrices are crucial so that the model can learn to produce “good” context vectors:

- W_q : Learns how to project word embeddings into the query space.
- W_k : Learns how to project word embeddings into the key space.
- W_v : Learns how to project word embeddings into the value space.

Attention Mechanism with Trainable Parameters



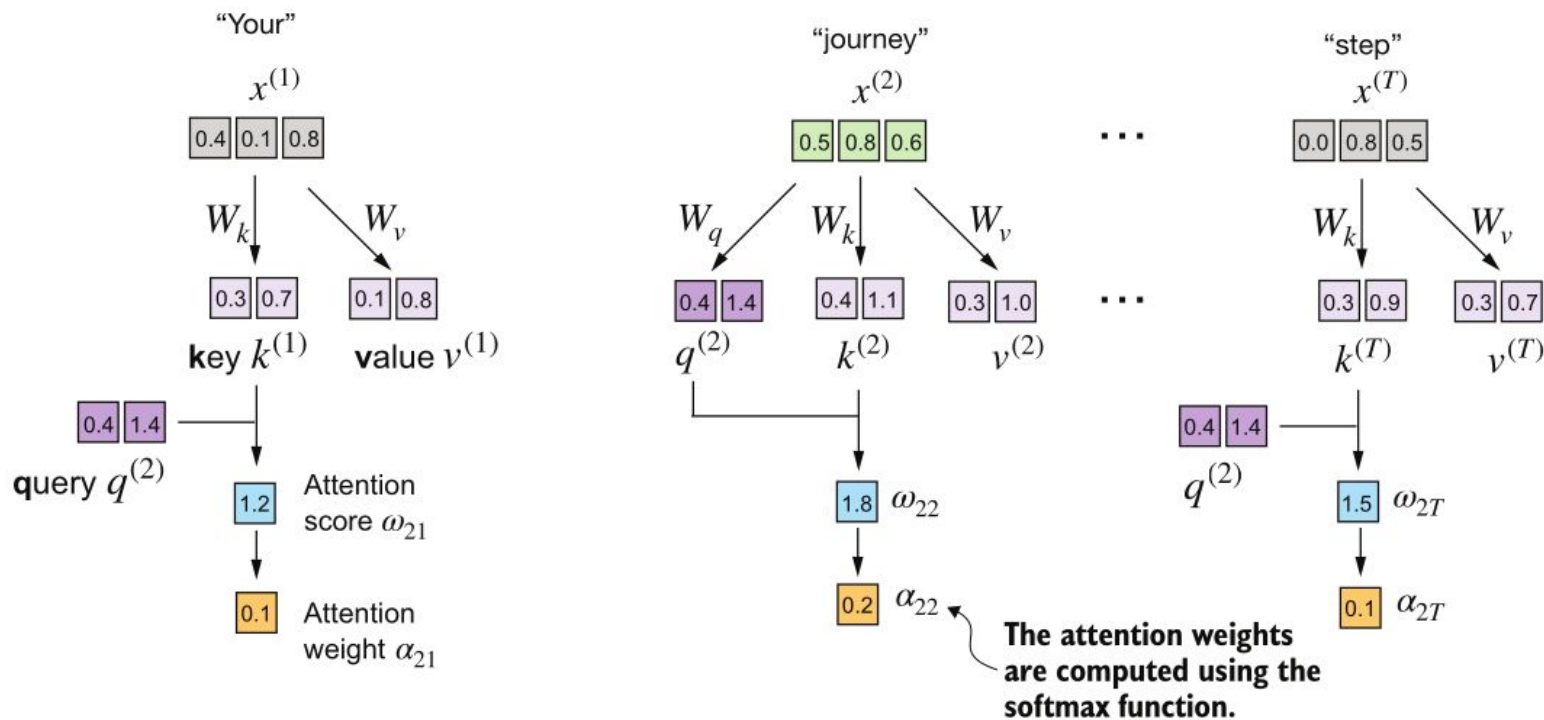
Attention Mechanism with Trainable Parameters



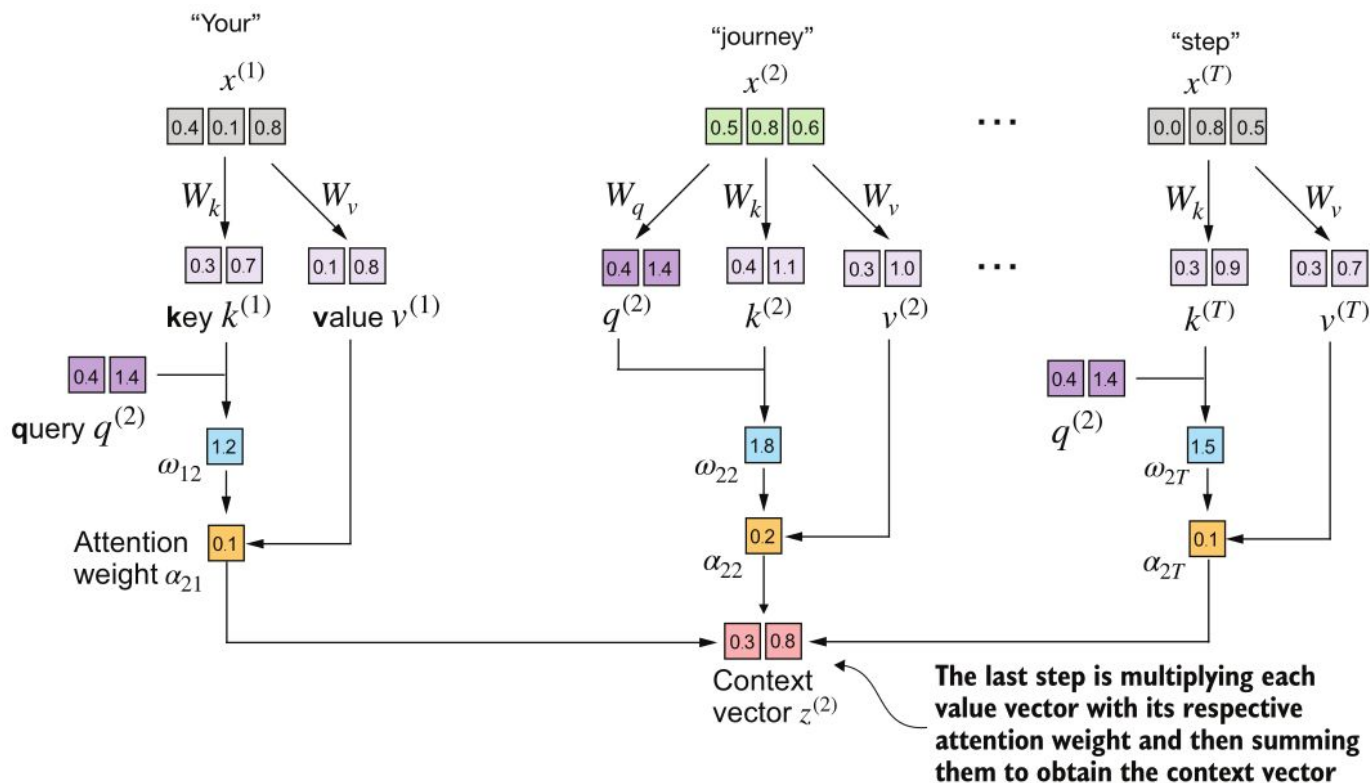
The unscaled attention score is computed as a dot product between the query and the key vectors.

Since we want to compute the context vector for the second input token, the query is derived from that second input token.

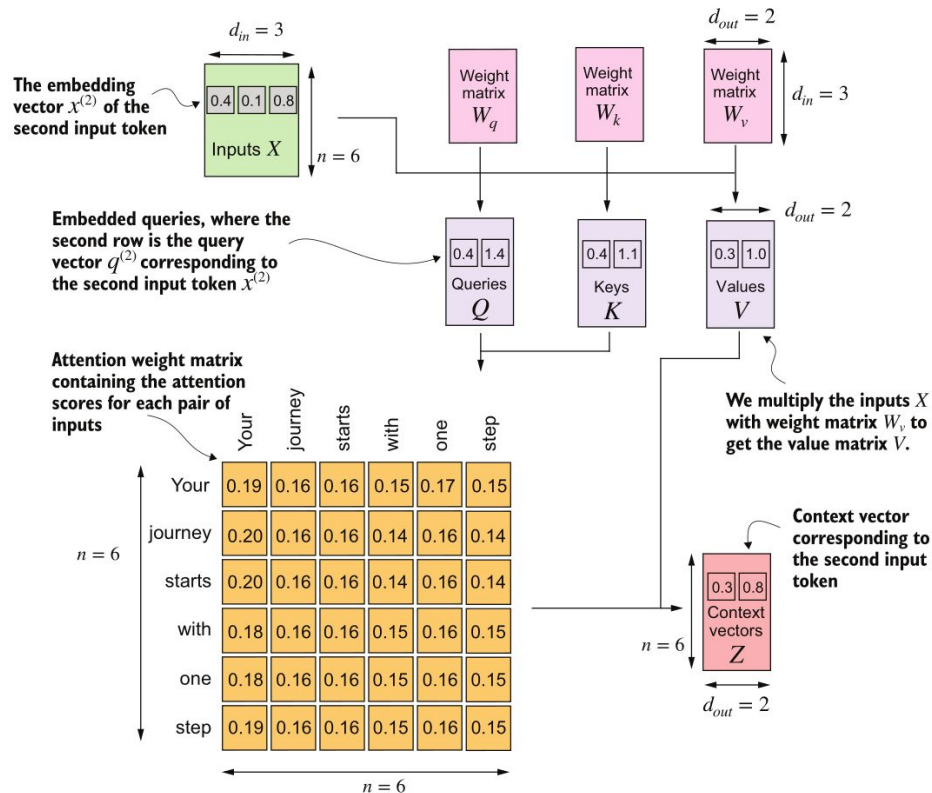
Attention Mechanism with Trainable Parameters



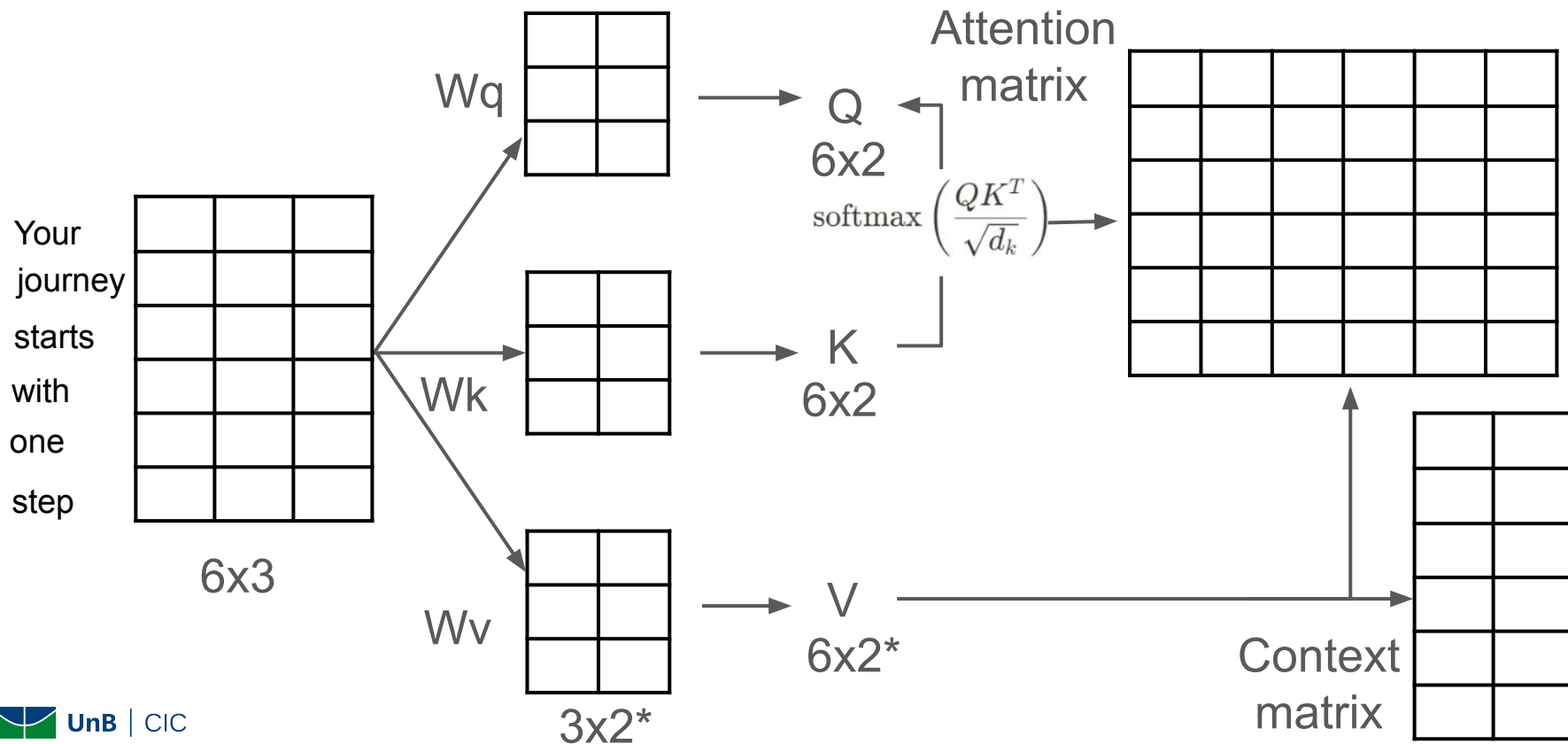
Attention Mechanism with Trainable Parameters



Attention Mechanism with Trainable Parameters



Attention Mechanism with Trainable Parameters



Attention Mechanism with Trainable Parameters

Should the attention matrix be the same for the encoder and the decoder?

	Your	journey	starts	with	one	step
Your	0.19	0.16	0.16	0.15	0.17	0.15
journey	0.20	0.16	0.16	0.14	0.16	0.14
starts	0.20	0.16	0.16	0.14	0.16	0.14
with	0.18	0.16	0.16	0.15	0.16	0.15
one	0.18	0.16	0.16	0.15	0.16	0.15
step	0.19	0.16	0.16	0.15	0.16	0.15

Encoders attend to both left and right contexts in order to learn the most informative embedding for representing a sentence.

Attention Mechanism with Trainable Parameters

Should the attention matrix be the same for the encoder and the decoder?

	Your	journey	starts	with	one	step
Your	1.00					
journey	0.55	0.44				
starts	0.38	0.30	0.31			
with	0.27	0.24	0.24	0.23		
one	0.21	0.19	0.19	0.18	0.19	
step	0.19	0.16	0.16	0.15	0.16	0.15

Decoders attend only to the left context, as their objective is next-token prediction. This mechanism is called causal attention (or masked attention), as future tokens are explicitly masked.

Attention Mechanism with Trainable Parameters

Should the attention matrix be the same for the encoder and the decoder?

	Your	journey	starts	with	one	step
Your	0.19	-Inf	-Inf	-Inf	-Inf	-Inf
journey	0.20	0.16	-Inf	-Inf	-Inf	-Inf
starts	0.20	0.16	0.16	-Inf	-Inf	-Inf
with	0.18	0.16	0.16	0.15	-Inf	-Inf
one	0.18	0.16	0.16	0.15	0.16	-Inf
step	0.19	0.16	0.16	0.15	0.16	0.15

A more efficient way to obtain the masked attention weight matrix in causal attention is to mask the attention scores with negative infinity values before applying the softmax function.

Attention Mechanism with Trainable Parameters

Should the attention matrix be the same for the encoder and the decoder?

	Your	journey	starts	with	one	step
Your	1.00	-Inf	-Inf	-Inf	-Inf	-Inf
journey	0.55	0.44	-Inf	-Inf	-Inf	-Inf
starts	0.38	0.30	0.31	-Inf	-Inf	-Inf
with	0.27	0.24	0.24	0.23	-Inf	-Inf
one	0.21	0.19	0.19	0.18	0.19	-Inf
step	0.19	0.16	0.16	0.15	0.16	0.15

A more efficient way to obtain the masked attention weight matrix in causal attention is to mask the attention scores with negative infinity values before applying the softmax function.

Attention Mechanism with Trainable Parameters

```
# For encoders [1, 6, 3] [batch_size, tokens, embedding]
def scaled_dot_product_attention(query, key, value):
    dim_k = query.size(-1)
    scores = torch.bmm(query, key.transpose(1, 2)) / sqrt(dim_k)
    weights = F.softmax(scores, dim=-1)
    return weights.bmm(value)

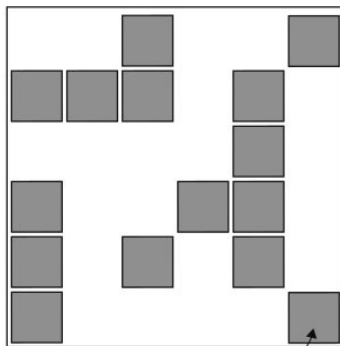
# For decoders [1, 6, 3] [batch_size, tokens, embedding]
def scaled_dot_product_attention(query, key, value):
    dim_k = query.size(-1)
    seq_len = query.size(1)
    scores = torch.bmm(query, key.transpose(1, 2)) / sqrt(dim_k)
    mask = torch.tril(torch.ones(seq_len, seq_len))
    scores = scores.masked_fill(mask == 0, float("-inf"))
    weights = F.softmax(scores, dim=-1)
    return weights.bmm(value)
```

Attention Mechanism with Trainable Parameters

It is common add dropout in the attention matrix. Dropout in deep learning is a technique in which randomly selected hidden layer units are ignored during training.

	Your	journey	starts	with	one	step
Your	1.0					
journey	0.55	0.44				
starts	0.38	0.30	0.31			
with	0.27	0.24	0.24	0.23		
one	0.21	0.19	0.19	0.18	0.19	
step	0.19	0.16	0.16	0.15	0.16	0.15

Attention weight for input tokens corresponding to "step" and "Your"



Dropout mask with random positions to be dropped

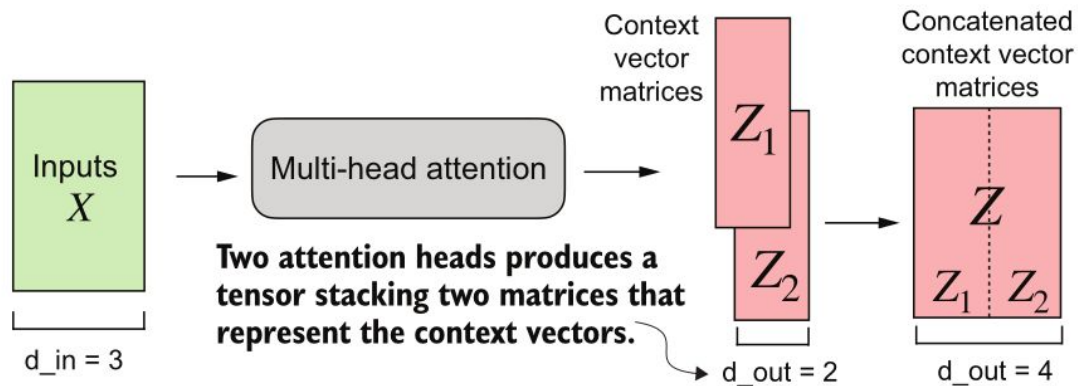
	Your	journey	starts	with	one	step
Your	1.0					
journey						
starts	0.38	0.30	0.31			
with		0.24	0.24			
one		0.19		0.18		
step		0.16	0.16	0.15	0.16	

The dropout mask applied to the attention scores will zero out certain attention scores

Multi-head attention

- Multi-head attention involves creating multiple instances of the self-attention mechanism.
- The main idea behind multi-head attention is to run the attention mechanism multiple times (in parallel) with different, learned linear projections

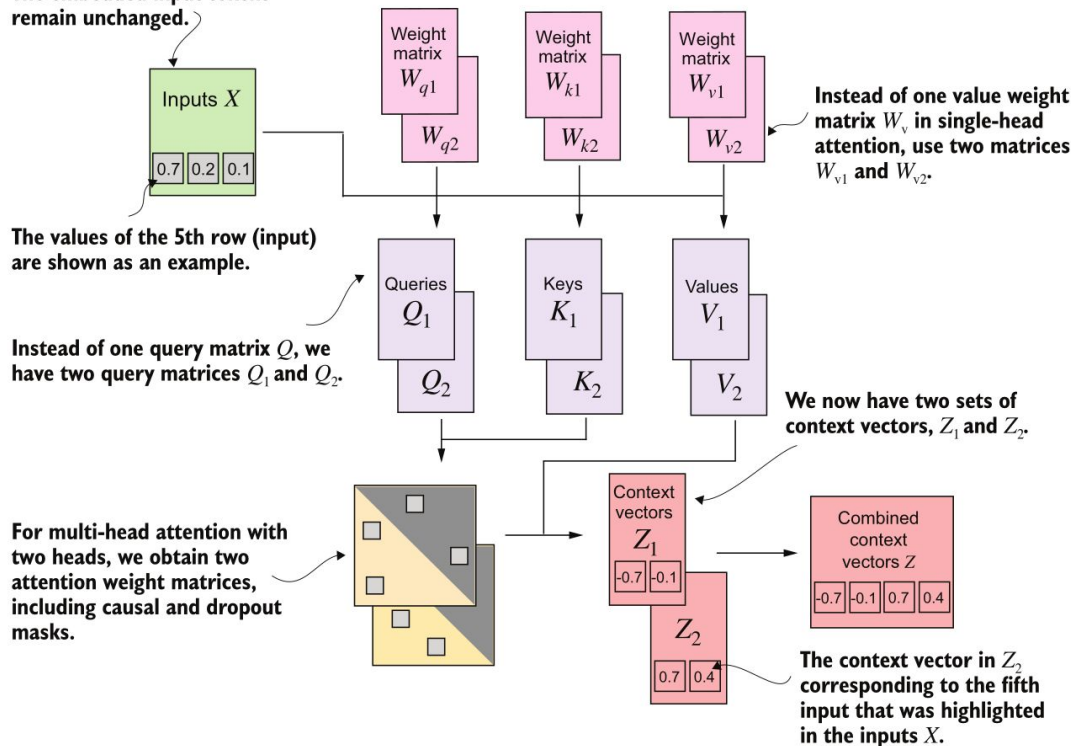
Multi-head attention



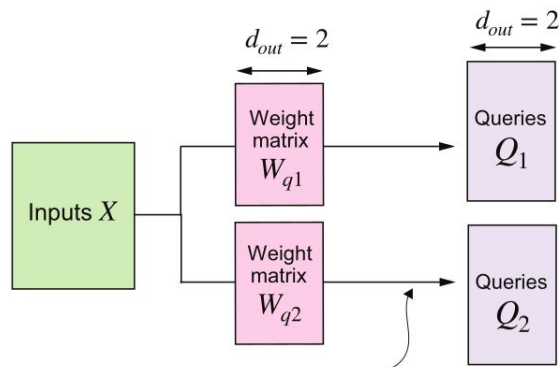
Choosing an embedding dimension of 2 ($d_{out} = 2$) for the context vectors results in a final embedding dimension of 4 ($d_{out} \times \text{num_heads}$).

Multi-head attention

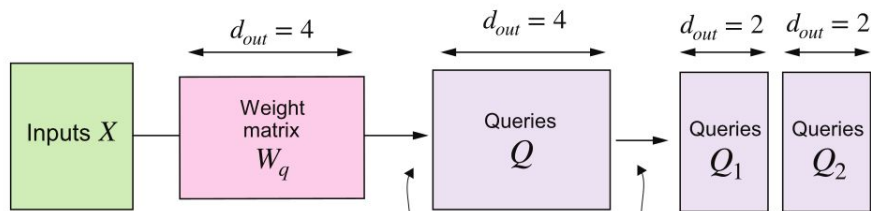
The embedded input tokens remain unchanged.



Multi-head attention



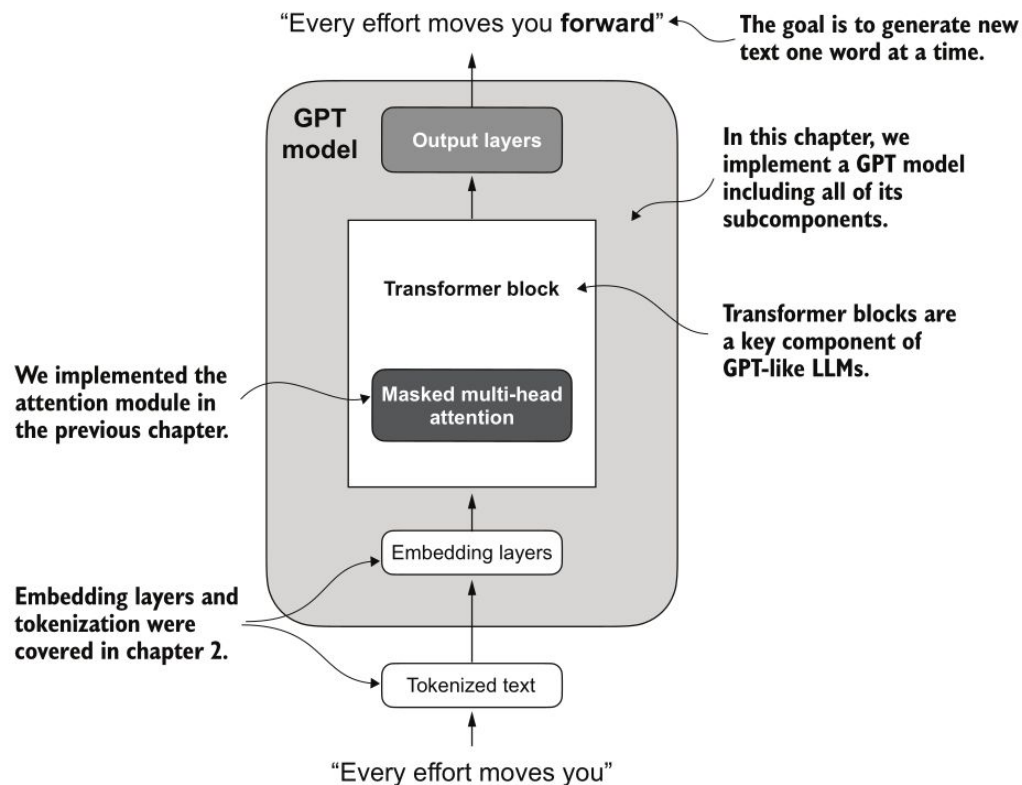
Perform two matrix multiplications to obtain the two query matrices, Q_1 and Q_2 .



Obtain queries Q with only one matrix multiplication.

Then split queries Q into Q_1 and Q_2 .

GPT Architecture



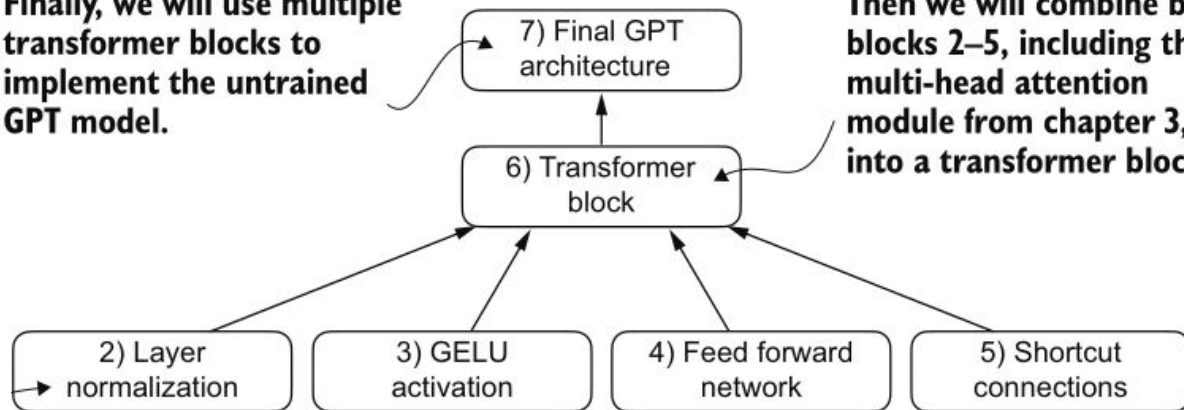
GPT Architecture

GPT-2 with 124M parameters has:

- Vocabulary size: 50257
- Context length: 1024
- Embedding dimension: 768
- Number of attention heads: 12
- Number of layers: 12

GPT Architecture

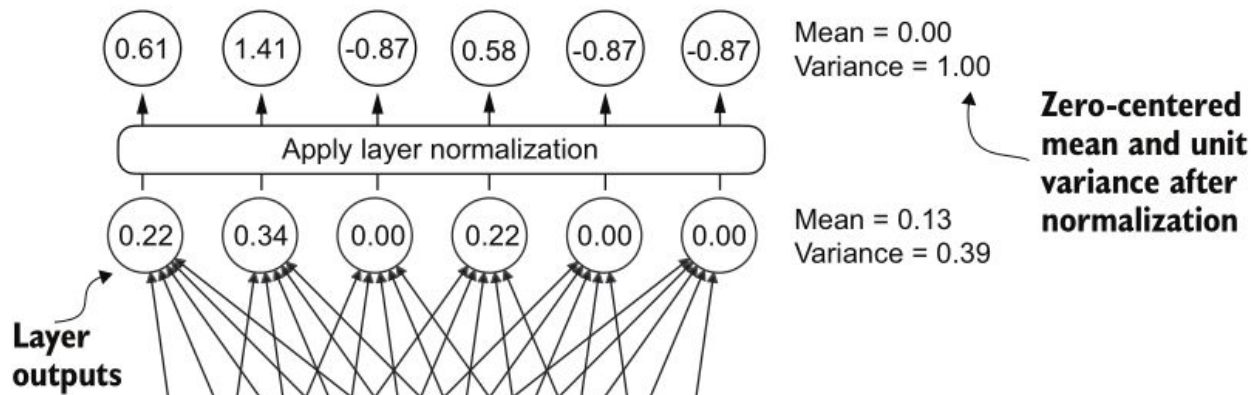
Finally, we will use multiple transformer blocks to implement the untrained GPT model.



Layer Normalization

- The main idea behind layer normalization is to adjust the activations (outputs) of a neural network layer to have a mean of 0 and a variance of 1;
- This avoid vanishing or exploding gradients;
- In GPT-2 layer normalization is typically applied before and after the multi-head attention module.

Layer Normalization

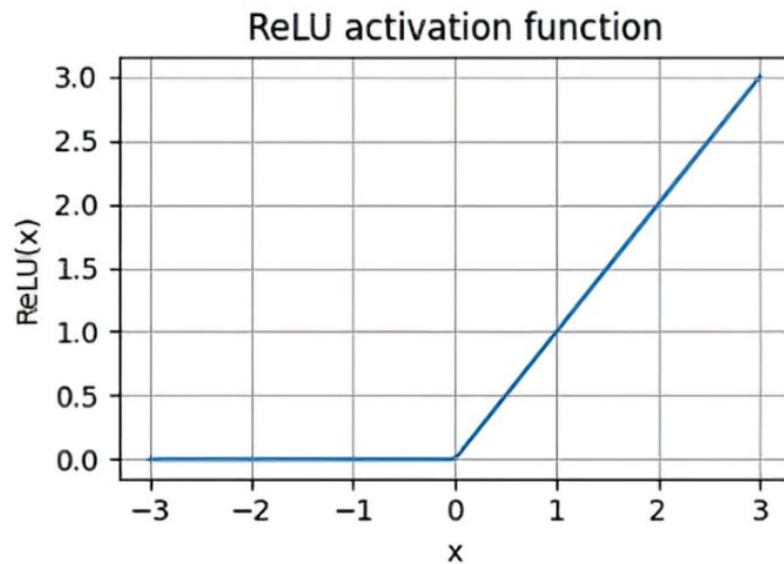
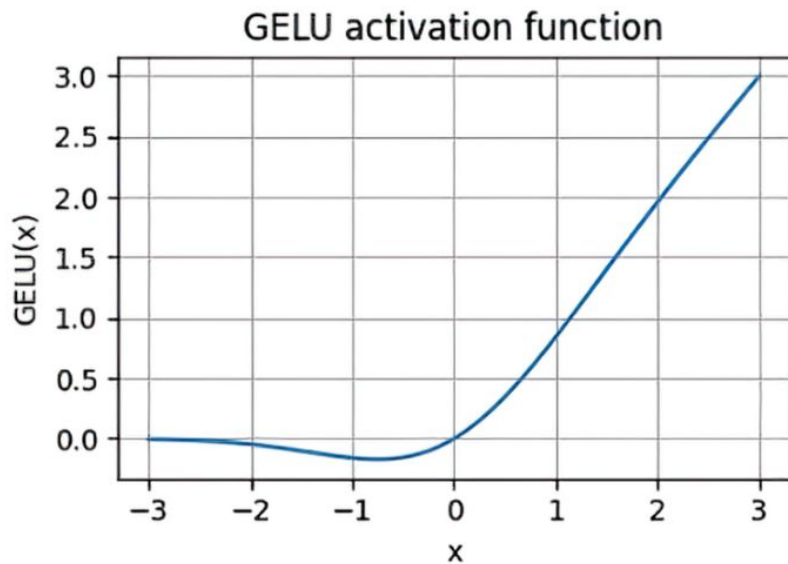


GeLU Activation

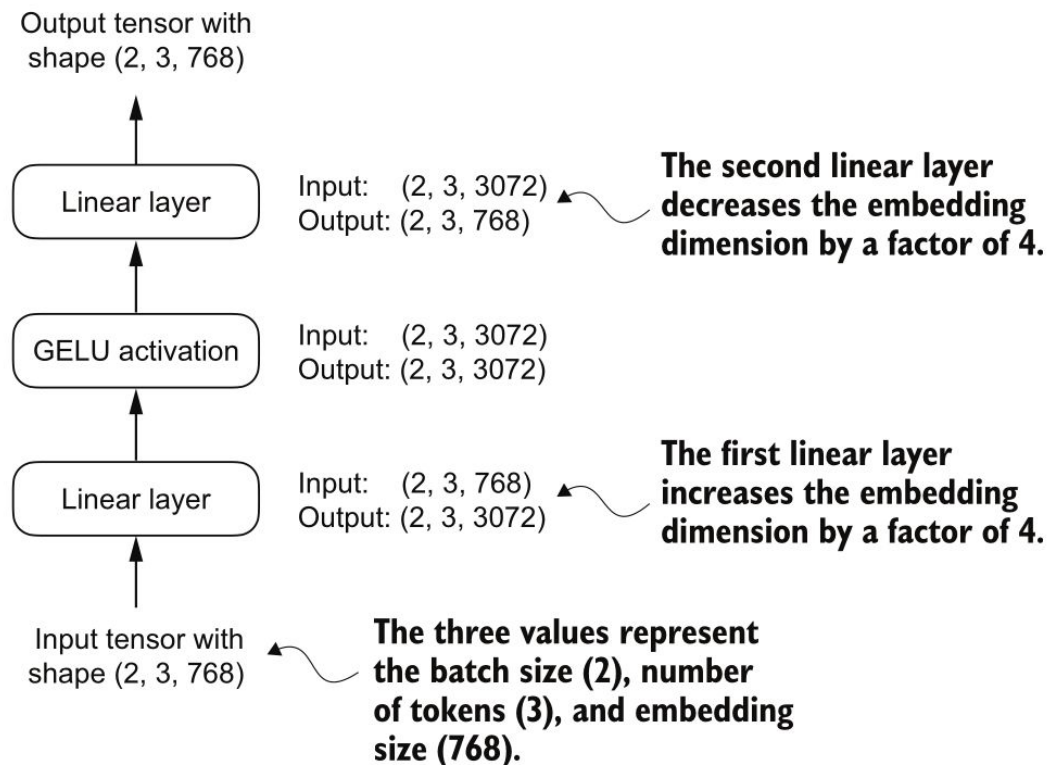
GELU and SwiGLU are more complex and smooth activation functions incorporating Gaussian and sigmoid-gated linear units, respectively. They offer improved performance for deep learning models, unlike the simpler ReLU.

$$GELU(x) \approx 0.5 \cdot x \cdot \left(1 + \tanh \left[\sqrt{\frac{2}{\pi}} \cdot (x + 0.044715 \cdot x^3) \right] \right)$$

GeLU Activation



Feed Forward Neural Network



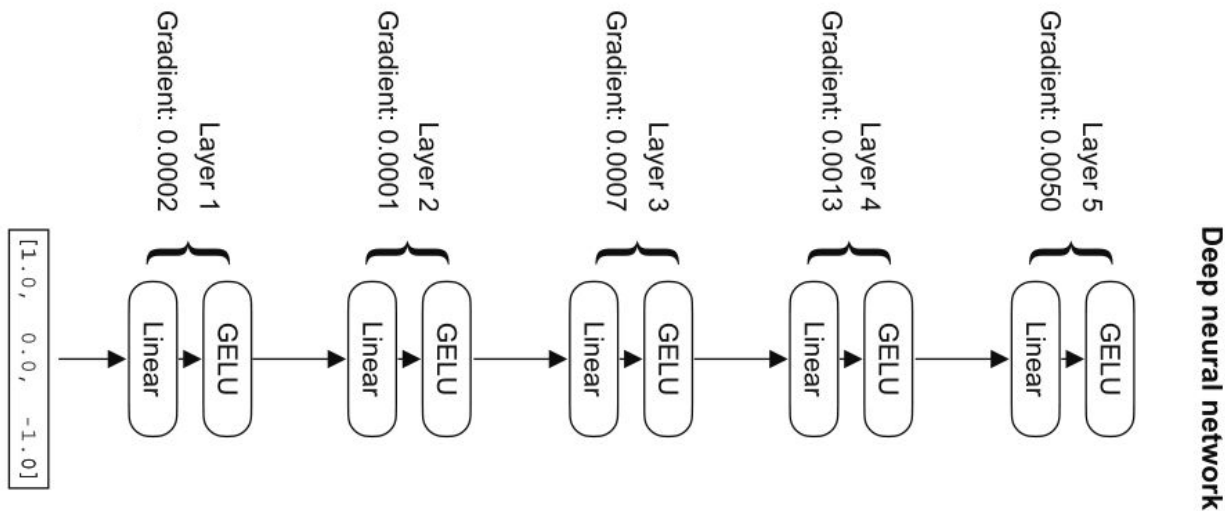
Shortcut Connections

Shortcut connections, also known as skip or residual connections:

- Shortcut connections were proposed to mitigate the challenge of vanishing gradients;
- A shortcut connection creates a shorter path for the gradient to flow through the network by skipping one or more layers;
- This is achieved by adding the output of one layer to the output of a later layer.

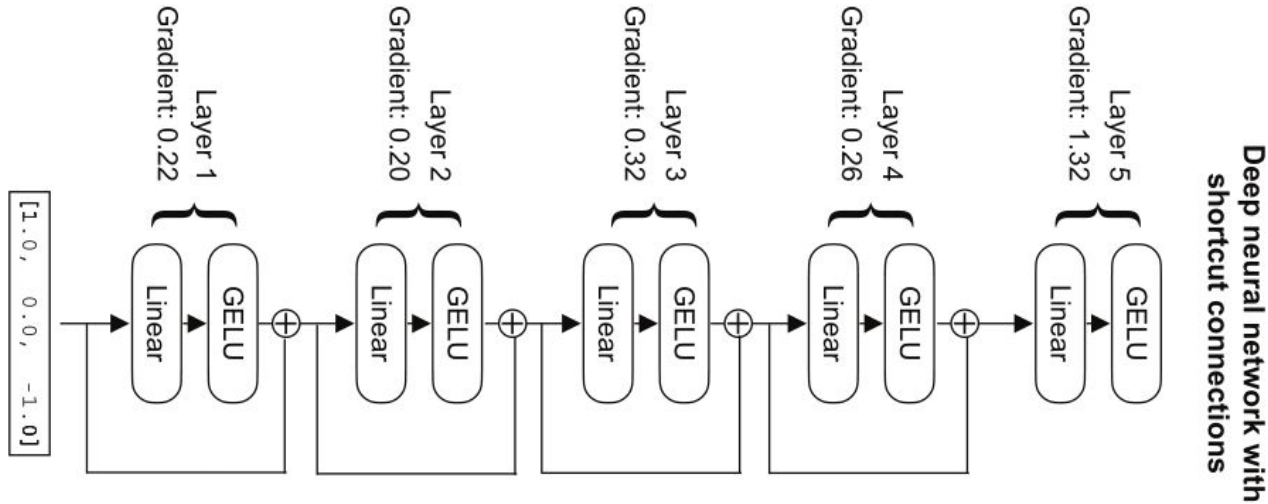
Without Shortcut Connections

In very deep networks, the gradient values in early layers become vanishingly small

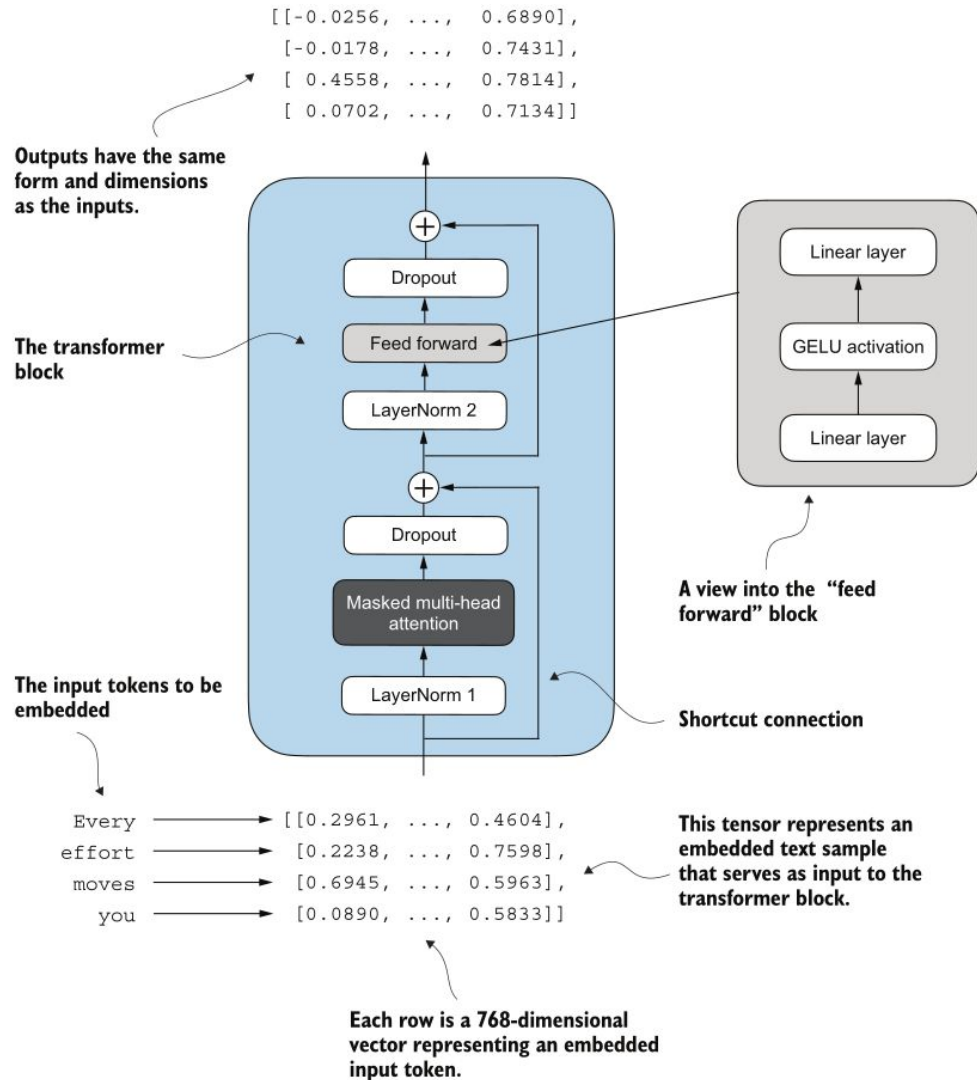


With Shortcut Connections

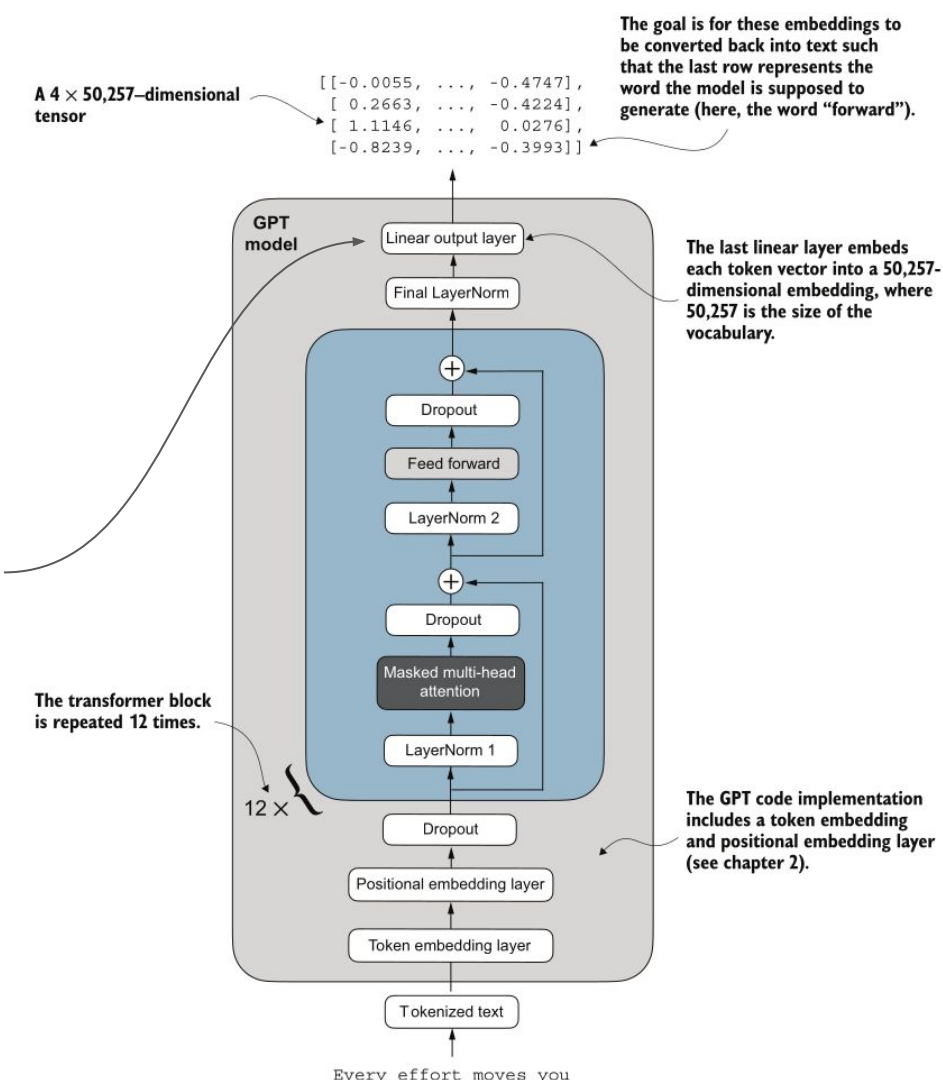
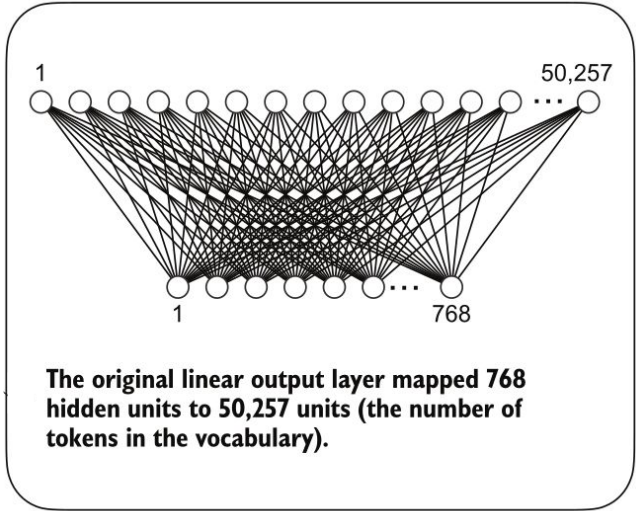
The shortcut connections help with maintaining relatively large gradient values even in early layers



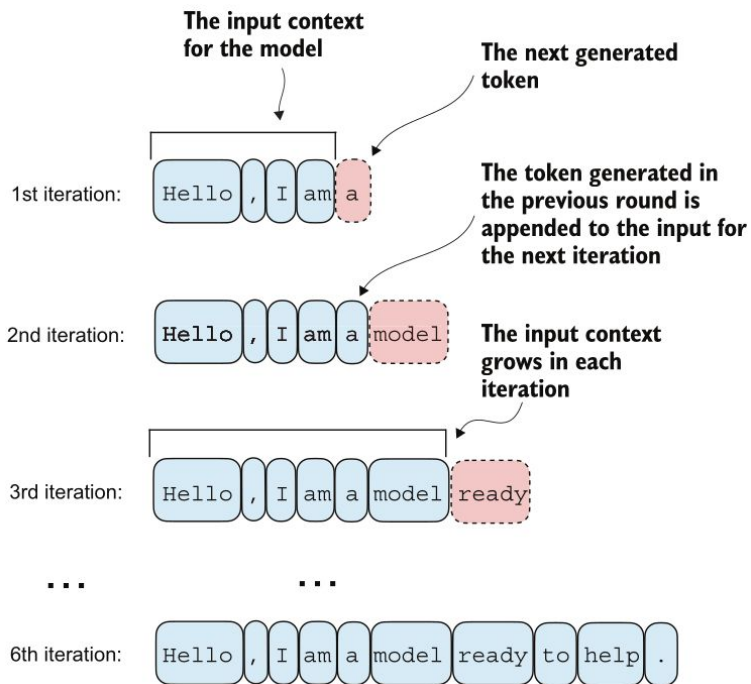
Transformer Block



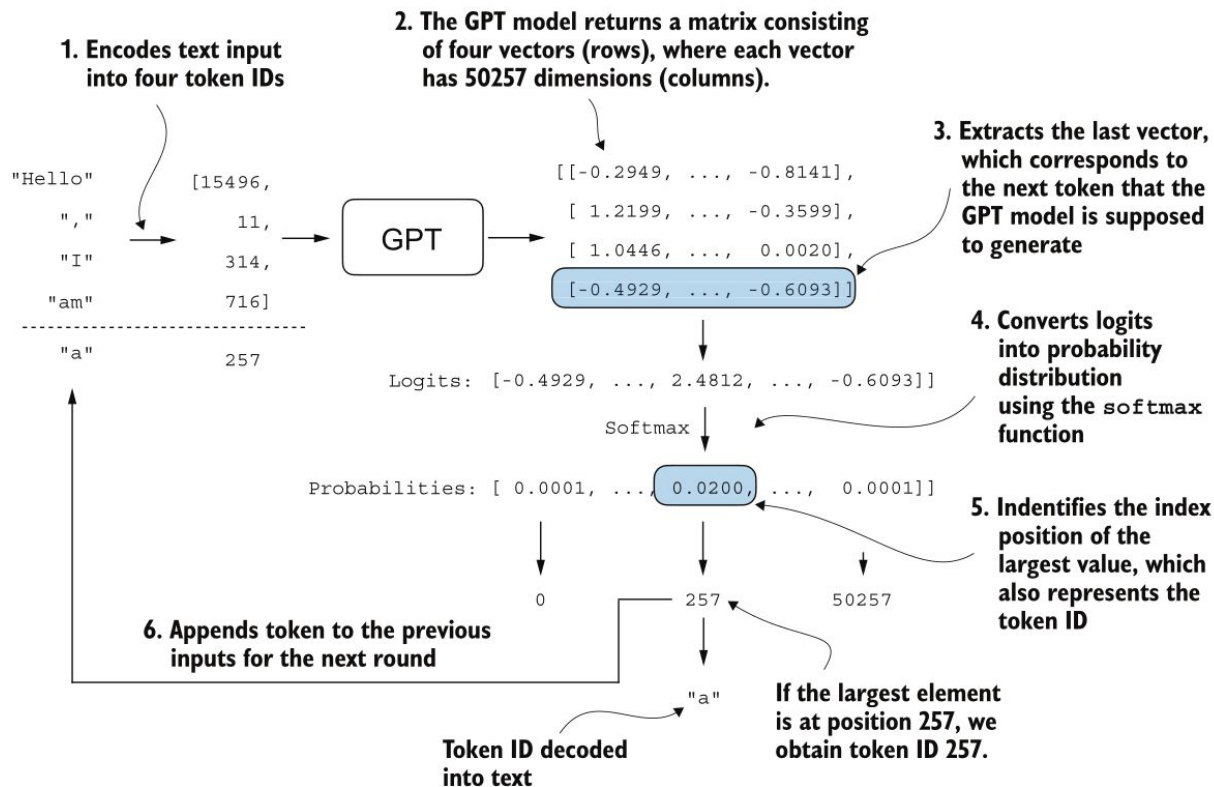
Final GPT Architecture



Generating text



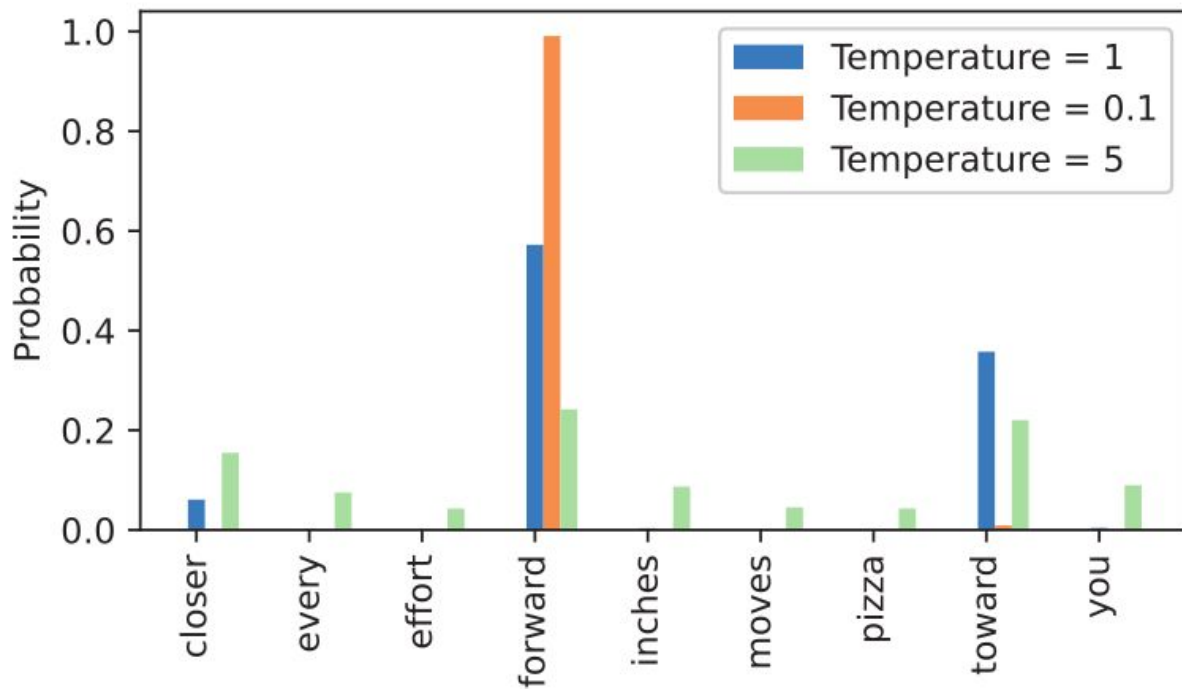
Generating text



Decoding strategies

- Greedy decoding: Sample the token with the highest probability as the next token using argmax
- Temperature scaling: Adds a probabilistic selection process to the next-token generation task (replaced the argmax with the multinomial) and divide the logits by a number greater than 0.

Temperature scaling



Top-k sampling

- Restrict the sampled tokens to the top-k most likely tokens and exclude all other tokens from the selection process by masking their probability scores.
- We can now apply the temperature scaling and multinomial function for probabilistic sampling to select the next token among these non-zero probability scores.

Top-k sampling

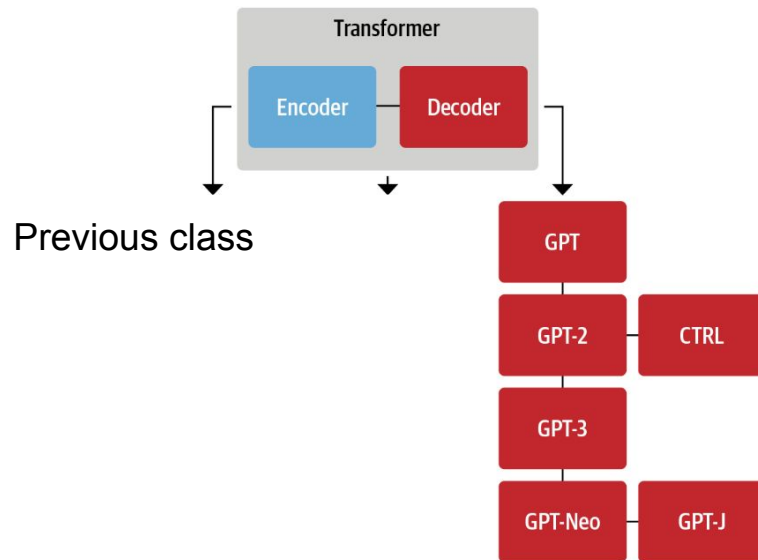
Vocabulary:	"closer"	"every"	"effort"	"forward"	"inches"	"moves"	"pizza"	"toward"	"you"
Index position:	0	1	2	3	4	5	6	7	8
Logits	= [4.51, 0.89, -1.90, 6.75, 1.63, -1.62, -1.89, 6.28, 1.79]								
↓									
Top-k (k = 3)	= [4.51, 0.89, -1.90, 6.75, 1.63, -1.62, -1.89, 6.28, 1.79]								
↓									
-inf mask	= [4.51, -inf, -inf, 6.75, -inf, -inf, -inf, 6.28, -inf]								
↓									
Softmax	= [0.06, 0.00, 0.00, 0.57, 0.00, 0.00, 0.00, 0.36, 0.00]								

By assigning zero probabilities to the non-top-k positions, we ensure that the next token is always sampled from a top-k position.

Top-k sampling

Restrict the sampled tokens to the top-k most likely tokens and exclude all other tokens from the selection process by masking their probability scores.

The Decoder Tree of Life



The Decoder Branch

The progress on transformer decoder models has been spearheaded to large extent by OpenAI.

Decoder model and its variants now a days:

- GPT;
- GPT-2 (small/large) (117 million/1.5 billion parameters);
- CTRL (based on control tokens);
- GPT-3 (175 billion parameters);
- ...

Conclusion

- Decoders generate text one token at a time, using previous outputs as inputs for future predictions.
- Unlike encoders, decoders use masked self-attention to focus only on the left context (past tokens).
- Masking future tokens with negative infinity values before softmax prevents the model from cheating during training.
- The model learns complex relationships by projecting embeddings into Query, Key, and Value spaces using trainable weight matrices (W_q , W_k , W_v).

Conclusion

Key Takeaway

- The decoder architecture, spearheaded by the GPT lineage, has revolutionized generative tasks by utilizing causal self-attention and massive parameter scaling to master the complexity of next-token prediction.

References

- RASCHKA, Sebastian. Build a large language model (from scratch). Shelter Island: Manning Publications, 2024.
- TUNSTALL, Lewis; VON WERRA, Leandro; WOLF, Thomas. Natural language processing with transformers: building language applications with Hugging Face. O'Reilly Media, 2022.
- RADFORD, Alec; NARASIMHAN, Karthik; SALIMANS, Tim; SUTSKEVER, Ilya. Improving language understanding by generative pre-training. OpenAI, 2018.